

Kutatást segítő ágens

Készítette: Jász Barnabás

Kivonat: A feladatom egy kutatás segítő program elkészítése volt, a programban ArXiv könyvtár segítségével egy MI ágensen keresztül lehet lekérdezni és letölteni publikációkat. Ezeket meg lehet tekinteni és egy másik MI ágens egy összefoglalót ír, ez alapján kiemeli a fontos szavakat a dokumentumból.

Kulcsszavak: Multiágens rendszer, Pydantic BaseModel, Phoenix Arize, Eszközhasználat, MCP

Tartalomjegyzék

Tartalomjegyzék.....	1
1. A feladat rövid ismertetése	2
2. Adatok.....	3
3. A megoldás részletes ismertetése.....	4
4. Futási példák	7
5. A megoldás értékelése	9
6. Hivatkozások.....	10

1. A feladat rövid ismertetése

Egy webapplikációt csináltam, ami képes LLM-alapú ágensek segítségével bármilyen kutatást segíteni. A felhasználó tud beszélgetni egy MI ágenssel, az ágensnek van 2 eszköze, amit ha úgy vél tud használni, hogy a felhasználó igényeit teljesítse. Az ágens tud az arXiv online könyvtáron keresztül keresni és letölteni dokumentumokat a megadott témában vagy a már letöltött publikációk között keresni kifejezésekre, témákra.

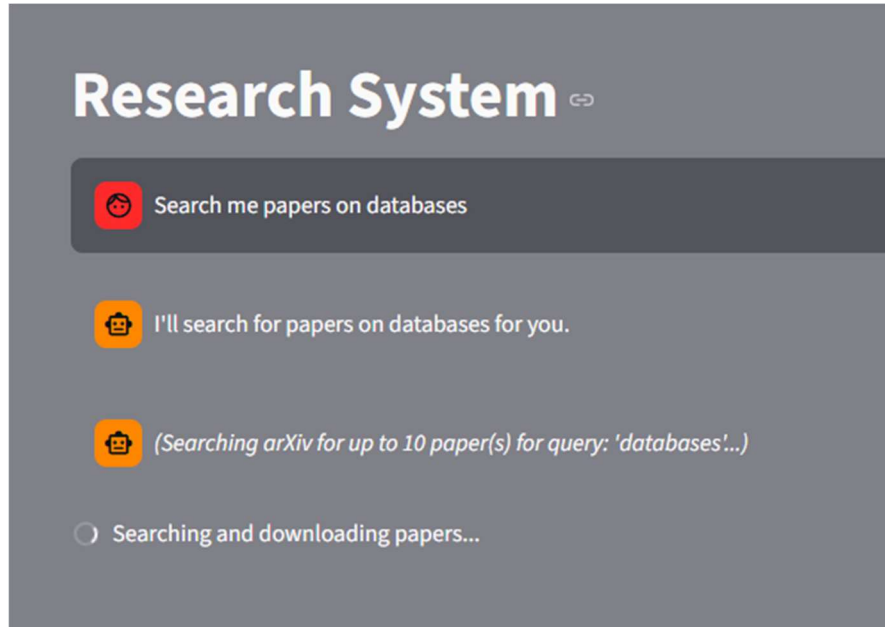
A letöltött anyagok menedzselésére egy teljesen különálló Files fül szolgál, ami egy digitális könyvtárként funkcionál. Itt a felhasználó egy tiszta, rendezett listában láthatja az eddig összegyűjtött összes publikációt, amelyeket egyetlen kattintással megnyithat olvasásra, vagy ha már nincs rájuk szüksége, törölheti is őket. Egy cikk kiválasztásakor a háttérben egy másik MI ágens lép működésbe, amely azonnal készít egy tömör, magas szintű összefoglalót a tanulmány főbb célkitűzéseiről és eredményeiről, így a kutatónak nem kell feleslegesen több tíz oldalt átnézni a lényeg megértéséhez. Ezt támogatja a beépített automatikus szövegkiemelő funkció is, amely egy kapcsoló segítségével aktiválható. Ez a modul szintén az LLM-et hívja segítségül: a szöveget kisebb darabokban elemzi, kiszűri a legfontosabb kulcsszavakat és kritikus mondatokat, majd ezeket vizuálisan, kiemelve jelöli meg a felületen megjelenő szövegben, radikálisan felgyorsítva a szakirodalom feldolgozását.

2. Adatok

ArXiv MCP szerveret használtam ahol rengeteg publikáció található meg.

3. A megoldás részletes ismertetése

A programot a Streamlit könyvtár segítségével egy webalkalmazásként csináltam meg. Ezt a keretrendszert már ismertem az önálló laboratórium miatt, így egyszerűen tudtam egy weboldalt létrehozni. Ebben a keretrendszerben már alapból van egy chat-es felület amit csak meg kellett hívnom.



A programomat 2 oldalra bontottam. Az egyik oldalon lehet az MI ágenssel beszélgetni, itt lehet kérni tőle, hogy kutasson különböző témákban, illetve a már letöltött dokumentumokban keressen. A keresést ArXiv MCP szerverrel oldottam meg, ahol rengeteg publikáció van és ezeket könnyen elérí egy LLM ágens. Az MCP szerver eléréséhez a FastMCP könyvtárat használtam.

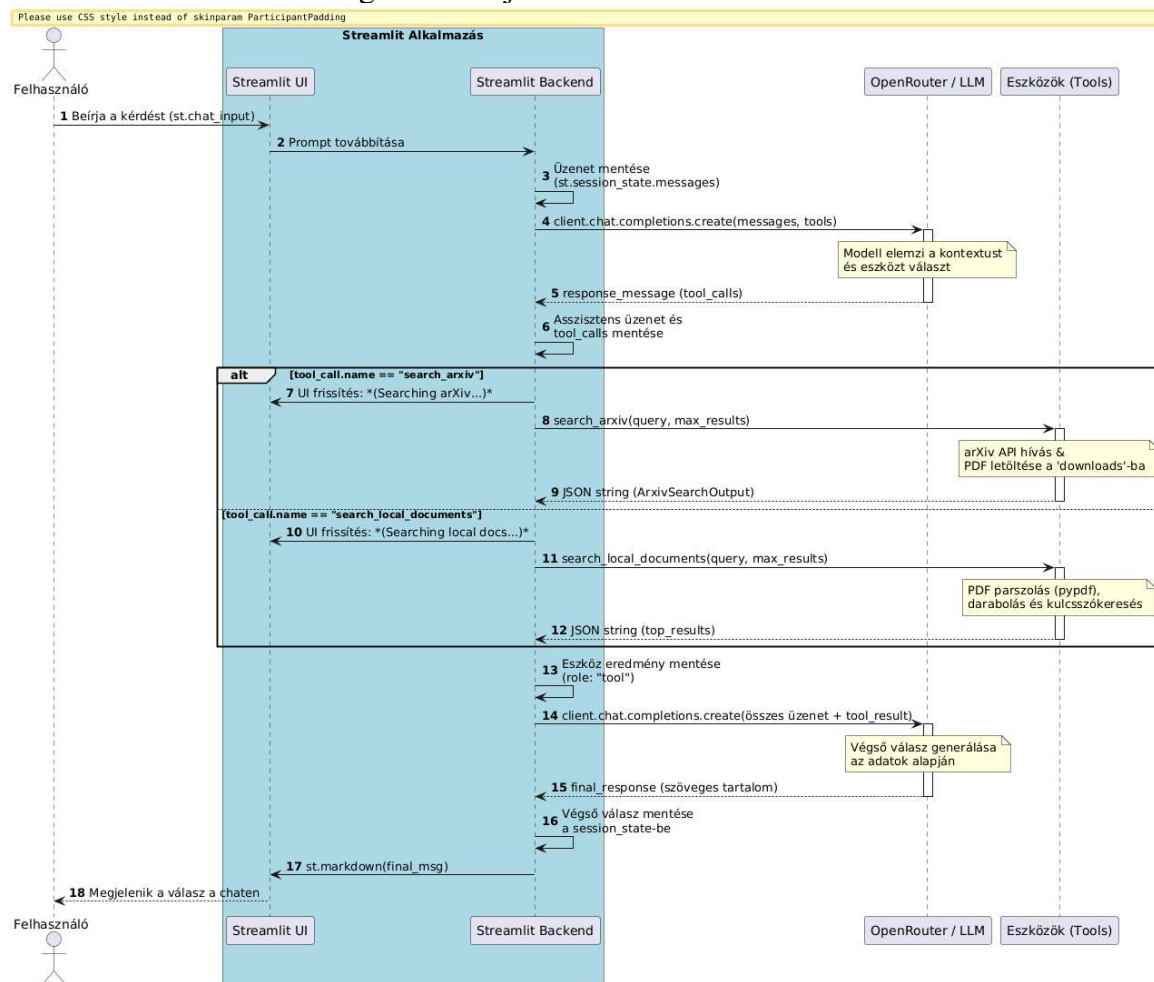
```
class ArxivPaper(BaseModel):
    id: str
    title: str
    authors: List[str]
    summary: str
    pdf_url: str
    local_pdf_path: str
    download_status: str

class ArxivSearchOutput(BaseModel):
    papers: Optional[List[ArxivPaper]] = None
    error: Optional[str] = None
```

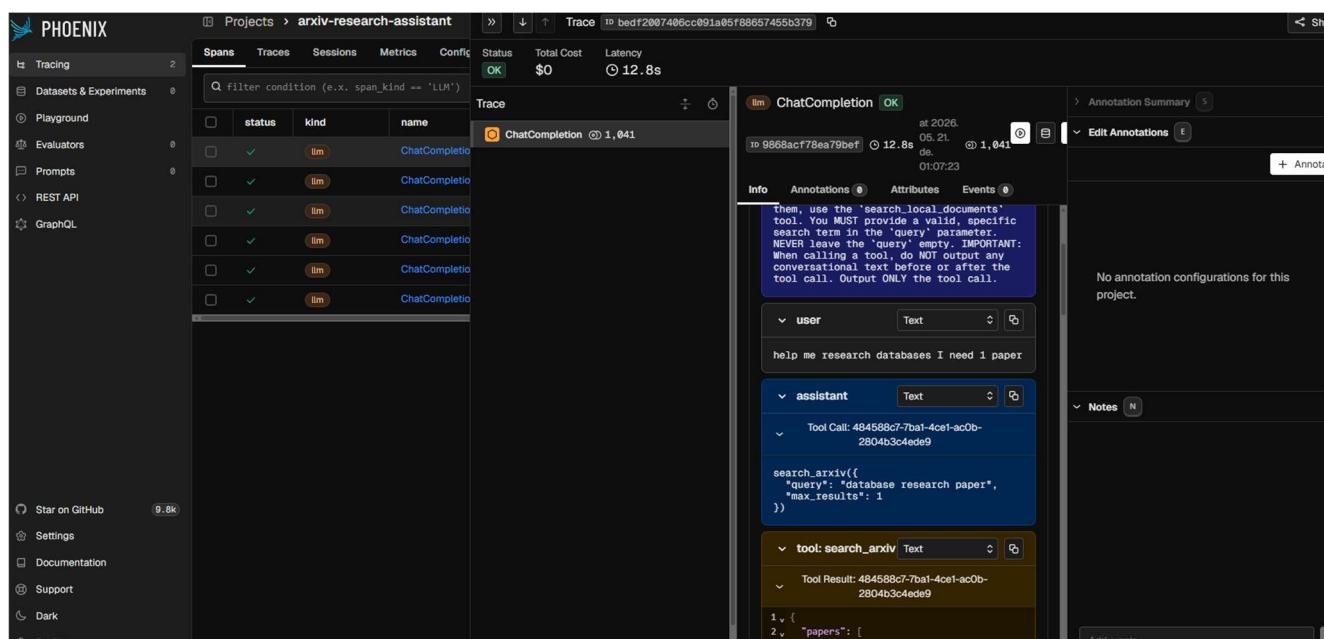
A publikációkat a Pydantic BaseModel segítségével rendezett struktúrájúvá tettem, ami segítette az LLM ágens választ. A keresés eszköz egy json formátumban adja vissza a megtalált dokumentumot vagy egy hibát.

A letöltött dokumentumok közti keresést egy RAG mechanizmussal oldottam meg. A letöltött publikációkat szétaraboltam, amihez a pypdf és a re könyvtárakat használtam. A kereséshez rangsorolom, hogy egy szövegrész mennyire releváns a kereséshez. A leginkább releváns találatokat ugyancsak json formátumban adja vissza az ágensnek, ha pedig hibára fut, akkor azt adja vissza.

A következő szekvenciadiagram mutatja be az eszközhívás menetet:



Az MI ágensek követésére az órán is tanult Phoenix-Arize programot használtam, itt tudtam követni az ágens eszközhívásait és a tokenhasználatot.

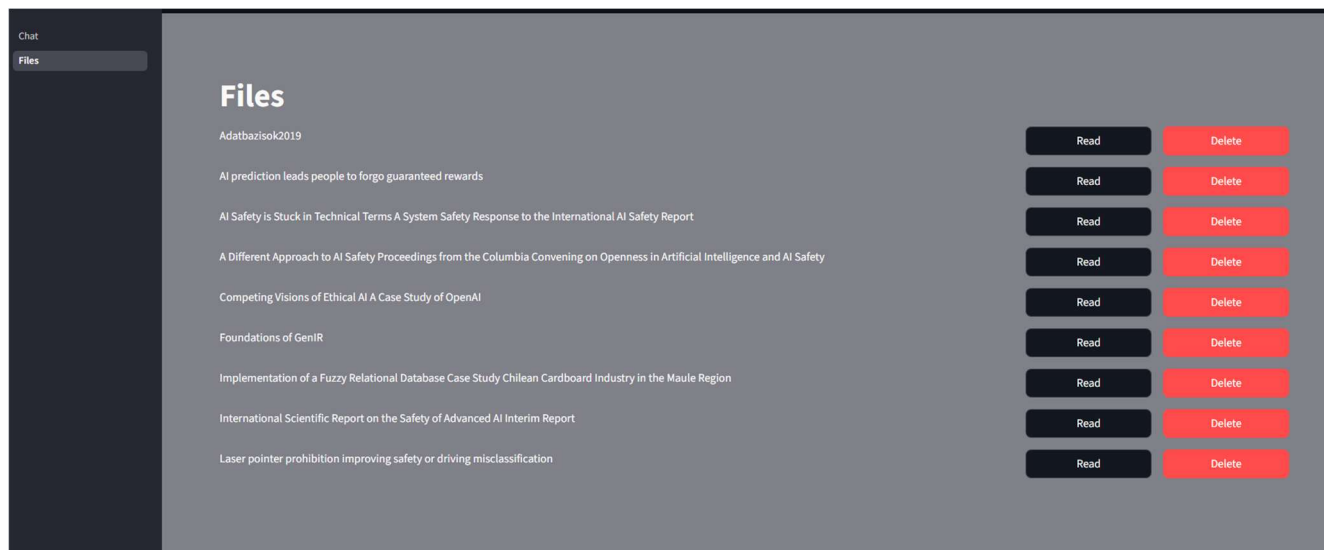


Először LLM-nek egy offline modellt próbáltam ki, ami a Llama3.2 volt, viszont ez nem teljesített jól az én eszközömmön. Utána Openrouter-en keresztül választottam az owl-alpha modellt-t, ez egy

ingyenes modell és kis méretéhez képest egészen jól használható a feladatra. A feladat végére sikerült a kvóta túllépést megoldani, így tudtam a tárgyból kapott OpenAi modelleket is használni, ezek szignifikánsan jobban teljesítettek, mint az owl-alpha, végül a gpt4o modellnél maradtam. Az MI-t OpenAI API-n keresztül hívtam meg és azon keresztül adtam át az eszközöket is.

A system prompt-ban meghatároztam azt, hogy mi a feladata és kikötöttem, hogy használja a rendelkezésre álló eszközeit, ha a felhasználó olyan utasítást ad, ehhez egy másik MI-t használtam, hogy pontos prompt-ot tudjak adni.

A Files oldalon meg tudjuk nézni a letöltött file-okat. A Files.py file-ban a dokumentumok kilistázási logikája található, itt vannak megtisztítva a file-ok nevei.



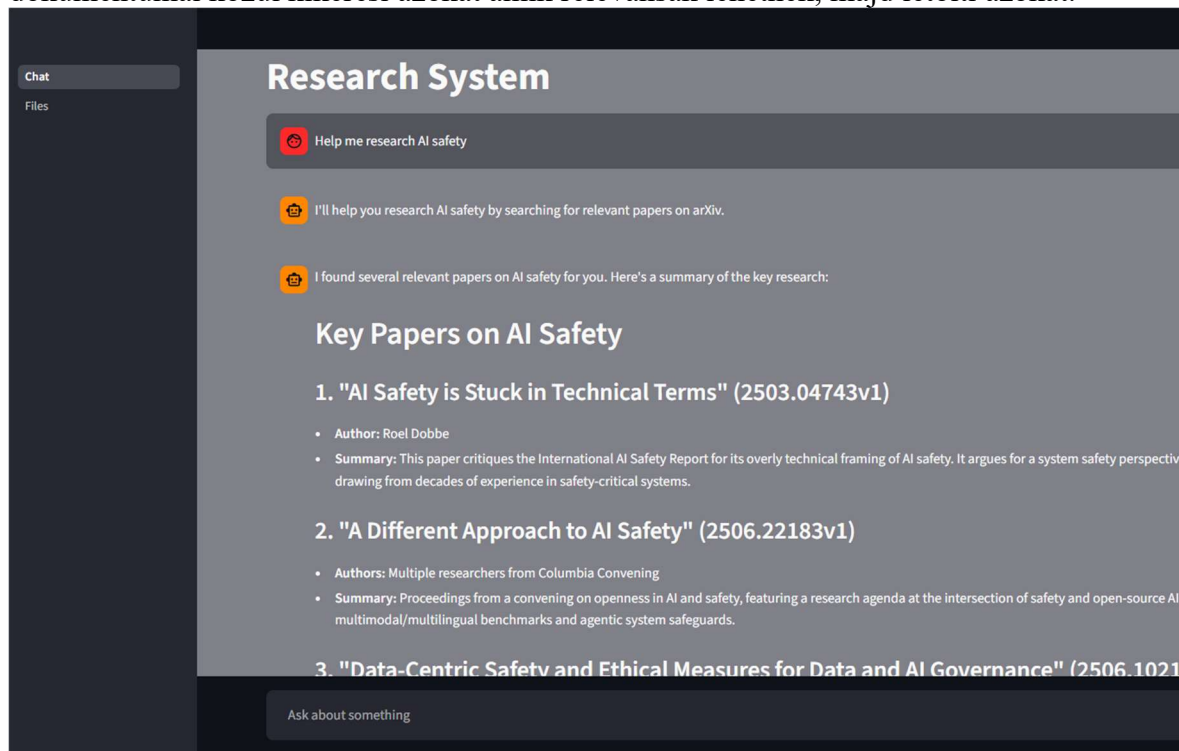
A Reading.py felel az olvasási logika kezelésére. Itt is BaseModel-t használok, hogy abba az osztályba gyűjtse a kiemelt részeket. Az összegzést és a kiemelést is LLM végez el, ezeket is először az owl-alpha modellel csináltam meg majd áttértem az gpt4o-ra. A kiemeléskor ugyancsak feldarabolva adtam oda a szöveget az LLM-eknek és így gyorsabb és költségkímélőbb lett. A program összegyűjti az összes kiemelendő szót a szövegrészből, majd ezeket megkeresi a szövegben és kiemeli.

Ahol tudtam, ott használtam párhuzamosítást a concurrent.futures könyvtárral használva, hogy ezzel is gyorsítsam az LLM-ek munkáját. Ha már egyszer megvolt a kiemelés és az összegzés is, azt elmentettem egy cache-be, hogy következő alkalommal ne kelljen megvárni, amíg lefut az LLM. Ha mégis le akarjuk futtatni még egyszer, akkor erre is van egy gomb, amivel a kiemelést le lehet futtatni újra.

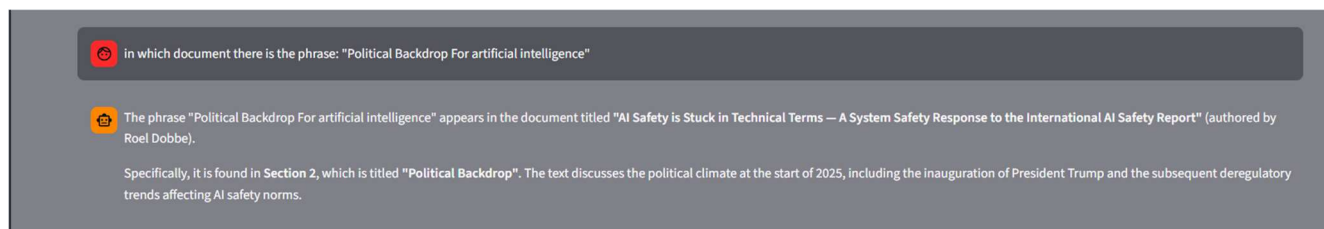
Ahol tudtam cache-eltem azokat az adatokat amikre még később szükség lehet, így is gyorsítva a rendszert, ezt a beépített streamlit cache taggal csináltam meg.

4. Futási példák

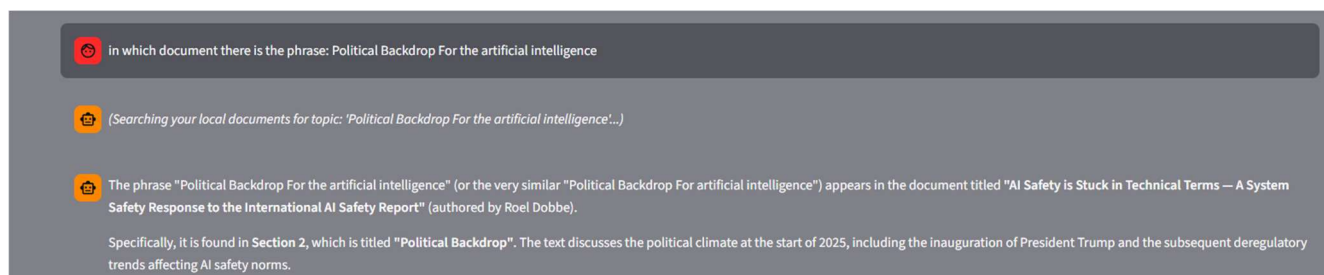
Ahogy belépünk az oldalra egy chat oldal fogad, ahol egy MI ágenssel tudunk beszélni, hogy miről szeretnénk kutatást indítani. Ha beírunk egy témát amiről szeretnénk kutatni akkor az ArXiv könyvtár dokumentumai közül kikeresi azokat amik relevánsak lehetnek, majd letölti azokat.



Ha a letöltött dokumentumok között szeretnénk keresni, akkor ezt is meg tudjuk kérdezni az ágensről.



Ha nem pontosan idézzük, akkor is megtalálja a megfelelő dokumentumot:



Ha el szeretnénk olvasni a letöltött dokumentumokat a files menüben ezt megtehetjük.



Ahogy rákattintunk egy dokumentum olvasására megjelenik egy MI összegzés, valamint ha szeretnénk be lehet kapcsolni a szöveg kiemelését, ezt egy kis idő múlva látjuk is.

Chat
Files

Files

Back to files list

AI Highlighting

Redo Highlighting

Summary:

Summary:

This paper examines the effectiveness of Australia's strict laser pointer regulations introduced in 2008, which prohibit the possession and importation of handheld laser devices exceeding 1 mW without a permit. Using Australia as a case study, the author investigates whether these laws have improved consumer safety or inadvertently driven misclassification and non-compliance. Analysis of over 40 laser pointers purchased legally in Australia—sourced via simple online searches—reveals widespread discrepancies between advertised and actual power output, misleading marketing, and poor adherence to safety standards. Findings suggest that suppliers have adapted to circumvent regulations, often selling high-powered lasers labeled as compliant devices, thereby undermining the law's safety objectives. The study concludes that the current regulatory framework has not enhanced public safety but instead fostered a market where prohibited devices remain easily accessible, posing ongoing risks to users and the public.

Reading: Laser pointer prohibition improving safety or driving misclassification

LASER POINTER PROHIBITION – IMPROVING SAFETY OR DRIVING MISCLASSIFICATION Paper #101 Trevor A Wheatley School of Engineering and Information Technology, UNSW Canberra, Campbell, ACT, 2612, Australia Abstract It is well known that since 2008 Australia has had some of the world's most restrictive laws regarding the possession and importation of "laser pointers" with powers exceeding 1 mW. Now four years on Australia is used as a test case and question whether this has actually improved safety for those wishing to purchase these devices or if it has impacted on the availability of prohibited devices. Results from the analysis of over 40 laser pointers legitimately purchased in Australia from local and international suppliers are presented. Specifically lasers that are readily available to everyday consumers through the simple on-line search "laser pointer 1mw" are targeted. The parameters investigated are quoted power versus measured power, correct representation in advertising and adherence to laser standards as related to specified use and purchase price. The analysis indicates that the suppliers in this market have learnt how to bypass the prohibition and the impact on general safety in these cases is detrimental. Introduction Laser pointer access and misuse is arguably one of the most emotive and controversial issues currently facing regulators, laser safety professionals and consumers. Many countries have ongoing problems with laser pointers being directed at aircraft, motor vehicles and members of law enforcement. Having had many conversations on the issue with various members of the laser community, it would seem that there is not a common opinion on how to remedy this situation. In fact there isn't even common definition of the term "laser pointer". Commonly the definition is related to the form factor and the power rating, but this is not always the case and not the case here. To avoid confusion with respect to the terminology, it is prudent to clarify the use of the term "laser pointer". For the purpose of this paper the term laser pointer refers to any hand-held battery operated laser device that produces a beam of any power that may be used for aiming, targeting or pointing. The approach taken by Australia in 2008 to mitigate this misuse was a hot topic for discussion in various forums and it is still a polarising issue within the laser community. Whether this approach was justified, fair, sensible, enforceable or correct is often the topic of these discussions, but opinions aside it was a proactive step to attempt to counter

Ha szeretnénk újra generálni a kiemelést, akkor a Redo Highlighting gombra kattintva ezt megtehetjük.

5. A megoldás értékelése

A projekt során egy teljeskörűen működő, intelligens kutatási asszisztenst sikerült összerakni, ami funkcionális téren teljesen működőképes. A rendszer integrálja a modern ágens-architektúrák alapköveit: a FastMCP-re épülő külső adatbázis-elérést, a letöltött PDF-ekből táplálkozó lokális RAG rendszert, valamint a strukturált adatkezelést. A Streamlitnek köszönhetően a felületet sikerült két jól elkülöníthető részre bontani, így a felhasználó egy letisztult chat-oldalon beszélhet az ágenssel, míg a Files menüpont alatt kényelmesen kezelheti, olvashatja vagy éppen törölheti a már letöltött publikációkat.

Az ágens teljesen önállóan dönti el, hogy az online arXiv könyvtárban kell-e kutatnia, vagy a meglévő állományokban. A helyi kereséshez ráadásul nincs szükség nehézkes, külső vektoros adatbázisokra; a pypdf és a reguláris kifejezések kombinációjával egy rendkívül kompakt, tiszta Python-alapú szövegrangsorolás jött létre. A nagyméretű dokumentumok feldolgozásának felpörgetése a ThreadPoolExecutor segítségével a program egyszerre, több szálon küldi be a szövegrészeket az LLM-nek, ami drasztikusan lefaragja a várakozási időt. Az adatok konzisztenciáját a Pydantic modellek garantálják, a beépített cache-mechanizmus pedig gondoskodik arról, hogy ne égessünk feleslegesen tokeneket és API-költségeket a már elemzett cikkeknél.

A működési tapasztalatok és a tesztek során ugyanakkor látni lehetett a rendszer korlátait is, amelyek leginkább a választott LLM modellek miatt vannak. A használt ingyenes owl-alpha modell néha hajlamos elengedni a system promptban kapott szigorú utasításokat. Emiatt az ágens időnként hibás formátumú JSON argumentumokat generál az eszközmeghívásokhoz, ami parszolási hibát szül, és megbénítja a folyamatot. Ehhez jön még hozzá a sebesség ingadozása is, mivel az ingyenes OpenRouter szerverek terheltségtől függően gyakran látványosan lelassulnak. A feladat végére sikerült implementálni az OpenAI modelleket így ki tudtam próbálni azok teljesítményét. A modellek jóval gyorsabban és pontosabban dolgoztak, mint az ingyenes modellek.

A másik kritikus pont az automatikus szövegkiemelés bizonytalansága. Mivel a modell a feldolgozás során egyszerre csak kisebb szövegrészeket lát, egyszerűen nem lehet szentírásnak venni, hogy a mesterséges intelligencia mindig a legrelevánsabb tudományos részeket szűrja ki, előfordulhat, hogy kulcsfontosságú összefüggések felett elsiklik, miközben lényegtelenebb mondatokat megjelöl. Végül a dizájn terén is kötni kellett kompromisszumokat. Habár funkcionálisan működik a program, esztétikailag és a UX szempontjából nem a legmegfelelőbb az oldal. Mindent összevetve azonban az alkalmazás funkcionálisan hibátlanul teszi a dolgát, a gyengeségek nagy része pedig egy robusztusabb, fizetős modellre való átállással szinte azonnal kiküszöbölhető.

6. Hivatkozások

- ☐ Nem használtam semmilyen generatív MI segédeszközt.
- ☒ Használtam generatív MI segédeszközt. Az MI-vel generált tartalmakat ellenőriztem, a generált kimenetek valóságtartalmáról meggyőződtem, az alábbi táblázatban megfelelően jelöltem minden használatot.

Felhasználási módok	Generatív MI eszköz(ök) neve	Érintett részek (fejezet, oldalszám, hivatkozás)	Használat becsült aránya (felhasználási módonként)
Irodalomkutatás	Gemini 3.1 Pro	Reading.py	2%
Prompt lényegi része	Mi a legjobb offline AI modell a saját rendszeremnek, ami elég gyors és fejlett *itt részleteztem a rendszeremet*		
Programkód generálása	Gemini 3.1 Pro	Reading.py, main.py	5%
Prompt lényegi része	Csináld meg a szöveg tisztítását és fűzd össze az összefüggő részeket (regex) többször is		
Új ötletek, megoldási javaslatok generálása	Gemini 3.1 Pro	Reading.py file	5%
Prompt lényegi része	Érdemes-e szálakat használni az offline modellek futtatásához és milyen könyvtárat használjak? Javasoljon system promptot egy ágenshez a feladathoz megfelelően		
Diagramok készítése	Gemini 3.5 Flash	Reading.py, Main.py, Files.py	1%
Prompt lényegi része	Dokumentációban használt szekvencia diagram készítése a kód alapján		
Egyéb (nevezze meg) Kisebb bugfixek	Gemini 3.1 Pro	Rading.py, Main.py, Files.py	3%
Prompt lényegi része	Javítsa ki a következő hibákat *hiba leírása*, hibakezelés		
Összesített százalékos érték (a feladat érdemi részére nézve)			16%
Összesített érték rövid, szöveges indoklása: A generatív MI-t nagy részben monoton feladatok vagy hosszabb nem feltétlen a tárgyhoz kapcsolódó információk keresésére használtam.			

Figyelem: a fenti százalékértékektől függetlenül az MI-vel generált minden tartalomért a szerző a felelős!